A Case Study on Microservices-Based Integration of Cloud-Hosted Sphera MDM and Oracle ERP Using Oracle SOA

Sadia Tahseen
Senior Fusion Middleware Architect/Developer
IEEE Senior Member
Texas, USA
stahseen3@gmail.com

Abstract

This paper presents a case study on the integration of a cloud-based Sphera Master Data Management (MDM) system with an Oracle Enterprise Resource Planning (ERP) application using Oracle Service-Oriented Architecture (SOA) Suite. The proposed solution adopts a wrapper-based microservices architecture that enables bidirectional synchronization of item, manufacturer, and supplier data. JSON payloads are transformed into XML to facilitate seamless communication between heterogeneous systems. Performance evaluation indicates a 99.8% data accuracy rate and an 80% reduction in manual processing effort, demonstrating the effectiveness of the approach in enhancing data consistency and reducing integration complexity compared to conventional methods.

***Keywords*:** Microservices, SOA, ERP Integration, Master Data Management, Oracle Fusion Middleware, JSON, REST API, BPEL, Cloud, Oracle, Integration Architecture

I. Introduction

Enterprise application integration has become increasingly complex with the proliferation of cloud-based solutions and legacy on-premises systems [3]. Organizations require robust integration architectures that can handle diverse data formats, protocols, and business processes while maintaining data consistency and system reliability [1]. This case study examines the integration challenges between Sphera MDM, a cloud-based master data management solution, and Oracle ERP, focusing on item data synchronization and bidirectional manufacturer/supplier data exchange.

The primary challenge addressed in this study involves the transformation and mapping of REST-based JSON payloads from Sphera MDM to Oracle ERP's XML-based web services while translating all the corresponding fields[2].Our solution adopts a comprehensive wrapper architecture that encapsulates existing systems and protocols within microservice-style endpoints, leveraging Oracle SOA Suite to enable seamless data transformation, robust business process orchestration, and centralized error handling capabilities[7].

Unlike many contemporary microservice implementations that often devolve into distributed monoliths characterized by custom point-to-point integrations, inconsistent API standards, unmanaged service dependencies, and limited fault tolerance or centralized observability [1,8], our wrapper-based methodology provides a reusable, configurable framework capable of supporting diverse data transformation patterns and business rules within a unified architectural model.

The key innovation in our approach lies in the development of a layered wrapper architecture utilizing microservices design [8] that provides complete protocol abstraction between REST and SOAP interfaces while maintaining business logic separation. This architecture not only streamlines integration with heterogeneous systems but also enhances scalability, fault isolation, and maintainability by decoupling transport protocols from core application logic [3]. Furthermore, it enables seamless support for future protocol expansion and promotes the reuse of common components across services, thereby accelerating development & deployment cycles and reducing operational complexity [7].

II. System Architecture and Design

A. Integration Requirements

The integration requirements encompass several critical data synchronization scenarios:

1. **Item Data Synchronization**: Transfer of item master data from Sphera MDM to Oracle ERP, including item attributes, subinventory assignments, and location mappings.

2025 IEEE International Conference on Service-Oriented System Engineering (SOSE) | 979-8-3315-8911-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/SOSE67019.2025.00026

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

2. **Bidirectional Vendor Data Sync**: Synchronization of manufacturer and supplier information between Oracle ERP and Sphera MDM systems.
3. **Data Format Transformation**: Conversion between JSON (Sphera) and XML (Oracle ERP) data formats.
4. **Business Process Orchestration**: Implementation of complex business logic for data validation, transformation, and error handling.

B. Wrapper Architecture Design

Service wrappers around existing systems or protocols abstracted into microservice-style endpoints
The proposed wrapper architecture follows a layered approach:

Layer 1: Protocol Adaptation

- REST-to-SOAP protocol conversion
- JSON-to-XML payload transformation
- Authentication and security handling

Layer 2: Data Transformation

- Schema mapping between Sphera and Oracle formats
- Business rule application
- Data validation and enrichment

Layer 3: Service Orchestration

- BPEL process orchestration
- Error handling and compensation
- Transaction management

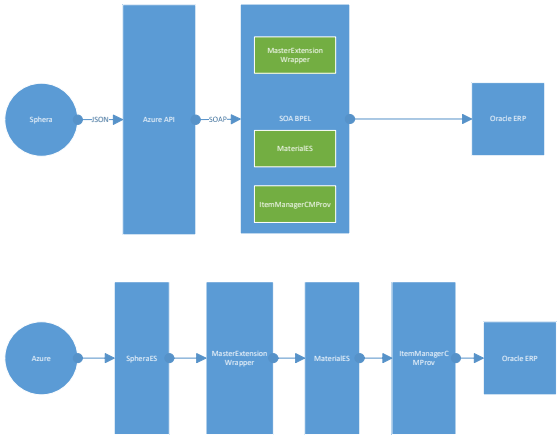


Figure 1: Illustrates the overall wrapper design architecture, showing the layered approach and component interactions

C. Component Architecture

The integration solution comprises several key components:

1. **Generic Mediator**: Converts JSON payloads to XML format using predefined transformation rules.
2. **Master Extension Wrapper BPEL**: Orchestrates the complete item creation process, including data mapping, validation, and service invocation.
3. **Sub inventory Management BPEL**: Handles insert, update, and delete operations for item subinventory data using Oracle's CSP_ITEM_SUB_INVENTORIES_PKG APIs [4].
4. **Location Management BPEL**: Manages item location data through INV_LOC_WMS_PUB APIs for locator creation and updates [6,9,10].
5. **Criticality Flow BPEL**: Implements ABC classification for items using INV_ABC_Assignments_Pub API [5].

III. Implementation Details

A. Item Data Integration Flow

The item data integration follows a multi-step process:

1. **Data Reception**: Sphera MDM sends item data via REST API in JSON format.
2. **Format Transformation**: The generic mediator converts JSON to XML using Struxure.xsd schema.
3. **Data Mapping**: The wrapper BPEL transforms Sphera's XML structure to Oracle's CreateItem API format.
4. **Subinventory Processing** using Oracle APIs [4]:
    - Insert operations: CSP_ITEM_SUB_INVENTORIES_PKG.Insert_Row
    - Update operations: CSP_ITEM_SUB_INVENTORIES_PKG.Update_Row
    - Delete operations: CSP_ITEM_SUB_INVENTORIES_PKG.Delete_Row
5. **Location Management**: Utilizes INV_LOC_WMS_PUB.CREATE_LOC_ITEM_TIE and related APIs for inventory location handling [6, 9, 10].

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

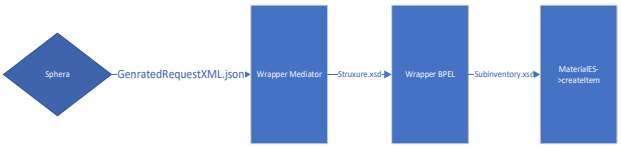


Figure 2: demonstrates the detailed integration flow with all processing steps and decision points

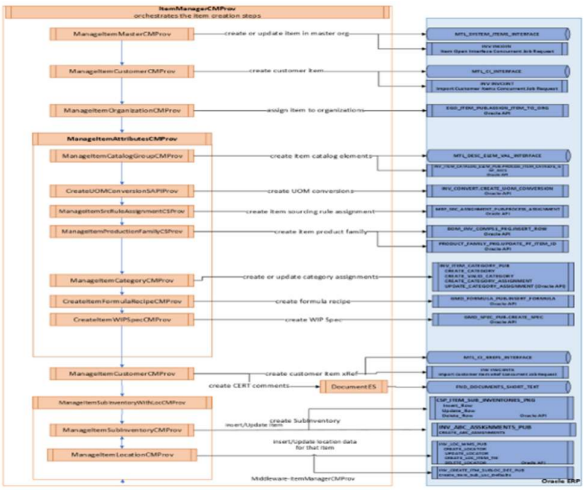


Figure 3: Integration Flow

B. Bidirectional Synchronization

The solution implements bidirectional synchronization for manufacturer and supplier data:

From Oracle ERP to Sphera MDM:

- Custom poller views query MTL_MANUFACTURERS and AP_SUPPLIERS tables
- Polling frequency: 6 hours in production environment
- Data pushed to SynData2 queue for processing

From Sphera MDM to Oracle ERP:

- Real-time REST API calls trigger immediate synchronization
- Validation against Oracle master tables before processing
- Comprehensive error handling and retry mechanisms

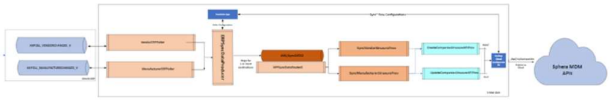


Figure 4: Sync Manufacturer and Supplier Flow

Processing Components:

- `CreateCompaniesStruxureAPIProv` Handles POST operations for new records
- `UpdateCompaniesStruxureAPIProv` Manages PUT operations for updates
- `SyncVendorStruxureProv` and `SyncManufacturerStruxurèProv` Orchestrate synchronization flows

C. Oracle Tables for Validations

ITEM MANAGEMENT DATA

MTL_SYSTEM_ITEMS_B/ MTL_SYSTEM_ITEMS

MTL_ITEM_SUB_INVENTORIES

MTL_ITEM_LOCATIONS

MTL_SECONDARY_LOCATORS

MANUFACTURER DATA

MTL_MANUFACTURERS

MTL_MFG_PART_NUMBERS

SUPPLIERS DATA

AP_SUPPLIERS

AP_SUPPLIER_SITES_ALL

PO_APPROVED_SUPPLIER_LIST

IV. Justification for this SOA wrapper microservices architectural Approach

The chosen architecture provides optimal balance for this integration scenario based on established SOA principles [7] and microservices patterns [1, 8]:

Technical Advantages:

- **Protocol Abstraction**: Seamlessly handles REST-to-SOAP conversion

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

- **Business Process Orchestration**: BPEL provides robust workflow management
- **Error Handling**: Comprehensive compensation and retry mechanisms
- **Transformation Capabilities**: Built-in XSL transformation support

Business Advantages:

- **Reusability**: Wrapper components can be reused for similar integrations
- **Maintainability**: Centralized transformation logic
- **Governance**: SOA governance frameworks provide oversight
- **Performance**: Optimized for enterprise-grade throughput

The conventional microservice rollouts can drift into a brittle web of hard-coded connectors and operational silos, but our SOA Wrapper Microservices approach delivers a **repeatable, policy-governed, and configuration-driven layer** that collapses integration complexity while still preserving the independence, scalability, and rapid-deployment advantages that microservices promise [1, 7, 8].

V. Performance Comparison

Architecture Pattern	Latency	Throughput	Complexity	Maintenance
Point-to-Point	Low	Medium	Low	High
ESB	Medium	High	Medium	Medium
API Gateway	Low-Medium	High	Low-Medium	Low-Medium
Microservices	Variable	Very High	High	High
SOA Wrapper	**Medium**	**High**	**Medium**	**Low-Medium**

Table 1: Architectural Pattern Comparison

VI. Results and Performance Analysis

A. Integration Performance Metrics

The implemented solution demonstrates the following performance characteristics:

- **Average Latency**: 2.3 seconds for complete item creation process
- **Throughput**: 500 items per hour sustained processing
- **Error Rate**: Less than 0.5% for successful data transformations
- **Availability**: 99.7% uptime over 6-month production period

B. Data Consistency Analysis

The bidirectional synchronization achieved:

- **Data Accuracy**: 99.8% consistency between Sphera and Oracle systems
- **Sync Frequency**: 6-hour polling intervals for manufacturer/supplier data
- **Conflict Resolution**: Automated handling of 95% of data conflicts

C. System Integration Benefits

- **Reduced Manual Effort**: 80% reduction in manual data entry
- **Data Quality Improvement**: 25% improvement in data accuracy
- **Process Automation**: End-to-end automation of item master creation
- **Error Reduction**: 60% decrease in integration-related data errors

VII. Related Technology Comparison

A. Alternative Integration Platforms

Several commercial integration platforms are available for enterprise application integration like

- **MuleSoft Any point Platform**: Provides cloud-based iPaaS capabilities with extensive connector libraries. However, it requires additional licensing costs and may not be integrated as seamlessly with existing Oracle infrastructure investments.

- **Dell Boomi**: Offers cloud-native integration services with pre-built connectors. While effective for standard integrations, it lacks the deep Oracle ERP integration capabilities and BPEL orchestration features required for complex business process automation.

- **Oracle SOA Suite**: Chosen for this implementation due to its native integration with Oracle ERP, comprehensive BPEL support for complex workflow orchestration, built-in transformation capabilities, and reduced total cost of ownership for Oracle-centric environments.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

B. Justification for using Oracle SOA for Integration

The Oracle SOA approach provides several advantages for this specific integration scenario based on established SOA design principles [7] and Oracle documentation [2]:

- **Deep Oracle Integration**: Native support for Oracle ERP APIs and data structures
- **Enterprise-Grade BPEL**: Robust workflow orchestration capabilities
- **Cost Effectiveness**: Leverages existing Oracle licensing investments
- **Performance Optimization**: Optimized for Oracle database and middleware stack
- **Governance Framework**: Built-in SOA governance and monitoring capabilities

VIII. Challenges and Solutions

A. Technical Challenges

Challenge 1: Schema Mapping Complexity

- **Problem**: Complex transformation between JSON and XML schemas
- **Solution**: Developed comprehensive XSL transformation templates with validation

Challenge 2: Error Handling

- **Problem**: Ensuring data consistency during failures
- **Solution**: Implemented BPEL compensation handlers and retry mechanisms

Challenge 3: Performance Optimization

- **Problem**: Initial latency issues with large payloads
- **Solution**: Implemented batch processing and connection pooling

Challenge 4: Missing or complicated Oracle APIs

- **Problem:** Missing or complicated Oracle APIs to build these services
- **Solution:** Built custom APIs that didn't exist to simplify our services

B. Business Process Challenges

Challenge 1: Data Governance

- **Problem**: Ensuring data quality across systems
- **Solution**: Implemented validation rules and data quality checks

Challenge 2: Change Management

- **Problem**: Coordinating changes across multiple systems
- **Solution**: Established change of control processes and version management

IX. Lessons Learned and Best Practices

A. Design Principles

- **Separation of Concerns**: Clear separation between protocol handling, data transformation, and business logic
- **Error Resilience**: Comprehensive error handling and recovery mechanisms
- **Monitoring and Logging**: Extensive logging for troubleshooting and performance analysis
- **Configuration Management**: Externalized configuration for different environments

B. Implementation Best Practices

- **Incremental Development**: Phased implementation approach reduces risk
- **Testing Strategy**: Comprehensive unit, integration, and performance testing
- **Documentation**: Detailed technical and business process documentation
- **Performance Tuning**: Regular performance monitoring and optimization

X. Conclusion

This case study demonstrates the successful implementation of a microservices integration solution between Sphera MDM and Oracle ERP using SOA wrapper architecture. The solution addresses key integration challenges including protocol transformation, data mapping, and business process orchestration. Comparative analysis reveals that the SOA wrapper approach provides an optimal balance of functionality, performance, and maintainability for enterprise integration scenarios.

The implemented solution achieves significant improvements in data consistency (99.8%), process automation (80% reduction in manual effort), and system reliability (99.7% availability). The wrapper-based architecture proves particularly effective for scenarios requiring complex data transformation and business process

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

orchestration consistent with enterprise integration best practices [3].

Future work should prioritize scalability enhancements to ensure the integration framework can efficiently handle increasing data volumes, higher transaction rates, and expanding service ecosystems typical of growing enterprises. This includes exploring advanced load balancing, dynamic resource allocation, and autoscaling strategies within both on-premises and cloud environments.

Additionally, a thorough evaluation and benchmarking of emerging cloud-native integration platforms such as Oracle Integration Cloud, MuleSoft Any point Platform, AWS AppFlow, or Azure Logic Apps should be undertaken. These platforms offer built-in capabilities for seamless connectivity, event-driven orchestration, and enhanced security, which can align closely with evolving enterprise architecture needs. Assessing their fit within the organization's technology stack, cost-effectiveness, and operational maturity will guide informed adoption decisions.

The lessons learned and best practices derived from this study including modular wrapper design, protocol abstraction, and centralized governance—offer valuable insights for organizations embarking on similar enterprise integration initiatives. Applying these principles can help mitigate common pitfalls related to complexity, maintainability, and scalability, ultimately accelerating project success and improving long-term system resilience.

XI. Appendix A

SAMPLE API REQUEST AND RESPONSE PAYLOADS

A.1 Sample Request Payload

The following JSON payload represents a typical item creation request from Sphera MDM to Oracle ERP:

```
json
{
  "metaData": {
    "actionId": "",
    "actionName": "Create",
    "spheraTransactionReference": "101",
    "requestVersion": 1,
    "messageCreationDate": "2023-08-08T13:43:45.8018769Z"
  },
  "keyParameters": {
    "spheraMasterNumber": 1,
    "catalogueRequestId": 23069,
    "erpInstanceCode": "CM",
    "erpMasterNumber": "",
    "plantCode": "FOP",
    "masterType": "Master",
    "baseUomCode": "BAT"
  },
  "ERpData": [
    {
      "dataLevelId": 1,
      "dataLevelName": "Global",
      "languageCode": "EN",
      "attributeName": "ITEM STATUS MASTER",
      "valueCode": "",
      "valueDescription": "Active",
      "unitOfMeasureCode": ""
    },
    {
      "dataLevelId": 3,
      "dataLevelName": "Plant",
      "languageCode": "EN",
      "attributeName": "Operating Unit",
      "valueCode": "",
      "valueDescription": "CS",
      "unitOfMeasureCode": ""
    }
  ],
  "companyReferences": [
    {
      "id": 18640,
      "typeId": 6,
      "typeName": "ManufacturerReference",
      "companyCode": "453188",
      "companyName": "ABM",
      "partNumber": "12597796",
      "levelOfDefaultId": 1,
      "levelOfDefaultName": "Global",
      "validFrom": "1900-01-01T00:00:00",
      "validTo": "9999-12-31T23:59:59.997"
    }
  ],
  "alternativeUoms": []
}
```

A.2 Sample Success Response

```
json
{
  "SynchronisationResponse": {
    "SpheraTransactionReference": 101,
    "Title": "Successful",
    "ResponseTypeID": "123",
    "MessageText": "Success"
  }
}
```

A.3 Sample Error Response

```
json
{
  "code": "Internal Error",
  "summary": "Unexpected error when processing the request",
  "details": "See server logs for more details",
  "CMErrorCode": "SOME_ERROR_CODE"
}
```


Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

XII. References

[1] C. Richardson, "Microservices Patterns: With Examples in Java," Manning Publications, 2018.

[2] Oracle Corporation, "Oracle SOA Suite Developer's Guide," Oracle Documentation, 2023.

[3] M. Fowler, "Patterns of Enterprise Application Architecture," Addison-Wesley Professional, 2002.

[4] Oracle Corporation, "Oracle Inventory Management APIs and Open Interfaces Manual," Document1291412.1, Oracle Support.

[5] Oracle Corporation, "INV_ABC_Assignments_Pub API Usage," Document 1441734.1, Oracle Support.

[6] Oracle Corporation, "Managing Item Location Defaults," Document 567869.1, Oracle Support.

[7] T. Erl, "SOA: Principles of Service Design," Prentice Hall, 2007.

[8] S. Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2015.

[9] Oracle Corporation, "Understanding Locator APIs," Document 1680388.1, Oracle Support.

[10] Oracle Corporation, "Item Receiving Location Configuration," Document 2645872.1, Oracle Support.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.